EXPERIMENT 6

InterProcess Communication using Shared Memory

OBJECTIVE:

- Learn and Understand InterProcess Communication using implementation of Shared Memory

BACKGROUND:

"Shared Memory Segments"

- A shared memory segment is a block (or chunk) of memory created using the shmget() system call.
- This segment has a unique ID (shmid) and can be attached to any process's address space using shmat().
- Once attached, processes can read/write to this shared memory just like normal memory — changes made by one process are instantly visible to others.

Shared Memory in InterProcess Communication (IPC) allows multiple processes to access the same memory space, enabling faster communication.
In traditional IPC methods (like pipes, FIFO, or message queues), data must pass through the kernel, involving **four copies**:

1. Server reads from file.
2. Server writes to IPC channel (to kernel buffer).
3. Client reads from IPC channel (from kernel buffer).
4. Client writes to output.

With **Shared Memory**, only **two copies** are needed:

1. Server reads data into shared memory.
2. Client reads from shared memory to output.

This makes shared memory **faster and more efficient** for process communication.

int shmget(key_t key, int size, int flags);
Allocates a shared memory segment.

- key is the key associated with the shared memory segment you want.
- size is the size in bytes of the shared memory segment you want allocated.
- The third parameter specifies the permissions on the shared segment i.e. read permission or write permission.
- The shmget function return a valid identifier which will be used in the next function shmat()

Return Values:
Upon successful completion, **shmget**() returns the positive integer identifier of a shared memory segment. Otherwise, -1 is returned

void *shmat(int shmid, const void *shmaddr, int shmflg);
Maps a shared memory segment onto your process's address space.

- shmid is the id as returned by shmget() of the shared memory segment you wish to attach.

- Addr is the address where you want to attach the shared memory. For simplicity we will pass NULL. NULL means that kernel itself will decide where to attach it to address space of the process.
- The third parameter will be zero if the value of the second parameter is NULL.

shmat() will fail if:
1. No shared memory segment was found corresponding to the given id.

Return Values:
Upon success, **shmat**() returns the address where the segment is attached; otherwise, -1 is returned and *errno* is set to indicate the error.

int shmdt(void *addr);

This system call is used to detach a shared memory region from the process's address space.

- Addr is the address of the shared memory

shmdt() will fail if:

1. The address passed to it does not correspond to a shared region.

Return Values:

Upon success, **shmdt**() returns 0; otherwise, -1 is returned and *errno* is set to indicate the error.
How to Delete Shared Memory Region:

For Deletion we will use **IPC_RMID** flag.

Return Values:
For IPC_RMID operation, 0 is returned on success; else -1 is returned.
Shmctl(shmid, IPC_RMID, NULL);

- shmid: The ID of the segment to delete
- IPC_RMID: Tells the OS to **mark the segment for deletion**
- NULL: No extra data needed

Post Lab Questions:

1. Create a private shared memory in C/C++. The process then creates a child and waits for the child to write the file's contents to shared memory. The parent then reads the shared memory and changes the case of each character and removes all integers from the data. The child reads it back and writes the changed data back to the same file. (The file name is passed as command line argument).
2. Create a C++/C program that creates a shared memory and waits for the other process to write n data of n number of Students. The process that created the shared memory then writes the data of students to the file.